

Informe de analisis del repositorio - Flappy Inverso

Analisis de historial Git, estructura del repositorio y evidencias del proyecto.

Nombre: Arnaldo Gonzalez
Repositorio: github.com/nalytox/bird-flappy
Fecha de entrega: domingo 05 de julio de 2026

Índice de Contenidos

1. Alcance de la actualizacion	1
2. Descripcion del proyecto	1
2.1. Funcionalidades actuales	1
3. Estructura actual del repositorio	2
4. Revision tecnica del codigo actual	3
4.1. Punto de entrada y estados del juego	3
4.2. Movimiento autonomo de Barranquin	3
4.3. Barranco, control y colisiones	3
4.4. Configuracion y parametros	3
5. Evolucion del desarrollo	3
6. Analisis del historial de Git	4
6.1. Commits relevantes	5
7. Evidencia del three-way merge	5
8. Resolucion manual de conflicto	6
9. Analisis de metricas del codigo actual	6
9.1. Interpretacion de metricas	6
10. Coherencia con el README	7
11. Reflexion final	7
12. Declaracion de uso de inteligencia artificial	7
13. Conclusion	7

Índice de Figuras

1. Evidencia local del historial mediante <code>git log --graph --oneline --all</code>	5
--	---

Índice de Tablas

1. Rol de los archivos principales.	2
2. Evolucion funcional del proyecto.	4
3. Commits relevantes observados en la evidencia.	5
4. Tamano aproximado de archivos actuales.	6

Índice de Códigos

1.	Estructura actual del repositorio.	2
----	--	---

1. Alcance de la actualizacion

Este informe corresponde a una version actualizada del analisis del repositorio **bird-flappy**. La actualizacion considera el enlace publico del repositorio y los archivos actuales entregados para el proyecto: **main.py**, **bird.py**, **gate.py**, **settings.py**, **utils.py**, **requirements.txt** y **README.md**.

La revision tiene como objetivo dejar el informe coherente con el estado actual del codigo: el proyecto ya no corresponde solo a una ventana con fondo, sino a un mini-juego funcional en Python y Pygame, con pajaro autonomo, barranco controlable, colisiones, puntaje y pantallas de inicio y termino. Para el historial de ramas y merges se usa como evidencia la captura local del comando **git log -graph -oneline -all**, incluida mas adelante.

2. Descripcion del proyecto

Flappy Inverso es una reinterpretacion del juego *Flappy Bird*. En la mecanica clasica, el jugador controla directamente al pajaro; en este proyecto se invierte la relacion: el pajaro vuela de manera autonoma y el jugador controla el hueco del barranco para evitar que choque.

El protagonista es **Barranquin**, inspirado en el loro trichahue (*Cyanoliseus patagonus bloxami*) y en la identidad visual asociada a la Universidad de O'Higgins. La ambientacion utiliza una paleta asociada a la precordillera semi-arida de la Region de O'Higgins y a colores del plumaje del trichahue.

El objetivo tecnico fue implementar un mini-juego funcional en **Python** usando **Pygame**, aplicando control de versiones con Git de forma incremental. El proyecto permite demostrar organizacion modular de codigo, uso de ramas, integracion de cambios, evidencia de merges y resolucion manual de un conflicto.

2.1. Funcionalidades actuales

A partir de los archivos actuales del repositorio, las funcionalidades implementadas son:

- Ventana de juego en Pygame, con resolucion de 400x640 y ejecucion a 60 FPS.
- Fondo visual con cielo degradado y capas de cerros, representando la precordillera de la Region de O'Higgins.
- Pajaro autonomo **Barranquin**, implementado en **bird.py** mediante la clase **Bird**.
- Movimiento vertical del pajaro mediante un oscilador amortiguado que persigue una altura objetivo.
- Seleccion periodica de la altura objetivo con **random.gauss()**, simulando una distribucion normal centrada en la mitad de la pantalla.
- Estela visual del pajaro para mostrar su trayectoria reciente.
- Barranco o compuerta controlada por el jugador, implementado en **gate.py** mediante la clase **Gate**.
- Control del hueco con teclado usando **flecha arriba**, **flecha abajo**, **W** y **S**.
- Deteccion de colisiones entre el pajaro y los rectangulos superior e inferior del barranco.

- Sistema de puntaje al atravesar correctamente el barranco.
- Dificultad progresiva: el hueco se reduce y la velocidad del barranco aumenta segun el puntaje.
- Pantallas de inicio y de fin de partida.
- Mejor puntaje guardado localmente en `best_score.json`, archivo que no se versiona.

3. Estructura actual del repositorio

La estructura actual del proyecto se organiza separando responsabilidades entre archivos. Esto permite que el codigo sea mas facil de mantener y explicar.

Código 1: Estructura actual del repositorio.

```

1 bird-flappy/
2 |-- .gitignore
3 |-- README.md
4 |-- requirements.txt
5 |-- settings.py
6 |-- utils.py
7 |-- bird.py
8 |-- gate.py
9 `-- main.py

```

Archivo	Funcion dentro del proyecto
<code>main.py</code>	Punto de entrada. Crea la ventana, maneja estados del juego, procesa teclado, actualiza entidades, calcula puntaje y dibuja cada cuadro.
<code>bird.py</code>	Define la clase <code>Bird</code> . Contiene el movimiento autonomo de Barranquin, la distribucion normal del objetivo vertical y el dibujo del pajaro.
<code>gate.py</code>	Define la clase <code>Gate</code> . Implementa el barranco, su desplazamiento, control por teclado, rectangulos de colision y funcion <code>circle_rect_collides</code> .
<code>settings.py</code>	Centraliza constantes de ventana, fisica, dificultad, colores y parametros visuales.
<code>utils.py</code>	Contiene la funcion <code>clamp()</code> , utilizada para limitar valores dentro de rangos seguros.
<code>requirements.txt</code>	Declara la dependencia principal del proyecto: <code>pygame>=2.5.0</code> .
<code>README.md</code>	Documenta el objetivo del proyecto, funcionalidades, instalacion, ejecucion y estructura.

Tabla 1: Rol de los archivos principales.

4. Revision tecnica del codigo actual

4.1. Punto de entrada y estados del juego

El archivo `main.py` coordina el proyecto. Importa `Bird`, `Gate` y `circle_rect_collides`, define los estados `start`, `playing` y `over`, y controla el bucle principal de Pygame. Tambien implementa el guardado simple del mejor puntaje mediante un archivo JSON local.

La logica de juego en `main.py` sigue una secuencia clara: leer eventos del teclado, actualizar el pajaro y el barranco, revisar colisiones, actualizar puntaje, regenerar el barranco si sale de pantalla y finalmente dibujar el fondo, las entidades y la interfaz.

4.2. Movimiento autonomo de Barranquin

El archivo `bird.py` implementa el comportamiento del pajaro. Barranquin no es controlado por el jugador: cada cierto intervalo aleatorio selecciona una altura objetivo mediante `random.gauss()`, centrada en `s.HEIGHT / 2`. Luego se mueve hacia ese objetivo mediante una formula de oscilador amortiguado:

$$a = k(y_{\text{objetivo}} - y) - cv_y \quad (1)$$

donde k corresponde a la fuerza del resorte y c a la amortiguacion. Esta decision evita que el movimiento quede sesgado permanentemente hacia abajo, como ocurriria con una gravedad constante, y permite que el vuelo se mantenga alrededor del centro de la pantalla.

4.3. Barranco, control y colisiones

El archivo `gate.py` define el barranco controlado por el jugador. El barranco se desplaza horizontalmente y su hueco puede moverse verticalmente con teclado. La dificultad aumenta mediante dos mecanismos: la velocidad del barranco crece con el puntaje y la altura del hueco disminuye progresivamente hasta un minimo definido.

La deteccion de colisiones se realiza mediante una funcion de choque circulo-rectangulo. Esta funcion calcula el punto del rectangulo mas cercano al centro del pajaro y luego compara la distancia cuadrada contra el radio del pajaro.

4.4. Configuracion y parametros

El archivo `settings.py` concentra los parametros mas importantes del juego. Entre ellos se encuentran la dimension de la ventana, la tasa de cuadros, los margenes de vuelo, la dispersion gaussiana, la velocidad del barranco, el tamano inicial y minimo del hueco, la velocidad maxima y la paleta de colores. Esta organizacion facilita ajustar el balance del juego sin modificar la logica principal.

5. Evolucion del desarrollo

El desarrollo fue organizado en etapas incrementales. La captura entregada del comando `git log --graph --oneline --all` muestra que primero se inicializo el proyecto, luego se agrego documentacion, despues se implemento el vuelo del pajaro, posteriormente se integro el barranco y finalmente se resolvió un conflicto en parametros de configuracion.

Etapa	Avance realizado
Inicializacion	Creacion de la ventana principal, configuracion base y fondo visual.
Documentacion inicial	Incorporacion de <code>README.md</code> para explicar objetivo, requisitos y ejecucion.
Vuelo de Barranquin	Implementacion del pajaro autonomo con objetivo vertical basado en distribucion normal.
Integracion del vuelo	Fusion de la rama del pajaro con <code>main</code> .
Barranco base	Implementacion inicial del barranco como obstaculo movil.
Control, colisiones y puntaje	Incorporacion del control con teclado, deteccion de choques y sistema de puntaje.
Ajustes de balance	Modificacion de parametros como velocidad del barranco y dispersion del vuelo.
Resolucion de conflicto	Unificacion del parametro de dispersion del vuelo en <code>settings.py</code> .

Tabla 2: Evolucion funcional del proyecto.

6. Análisis del historial de Git

Segun la evidencia local incorporada, el historial muestra **10 commits visibles** en el grafo. Este dato se reporta a partir de la captura del comando `git log --graph --oneline --all`. El repositorio publico sirve para verificar el codigo actualizado; el detalle de ramas se documenta con la evidencia local del historial.

- **Numero de commits visibles en la evidencia local:** 10.
- **Rama principal:** `main`.
- **Ramas de desarrollo usadas:** `feature/pajaro-random` y `feature/barranco-control`.
- **Archivo de conflicto evidenciado:** `settings.py`.
- **Commit final visible:** 807b58f, asociado a la resolucion del conflicto de dispersion del vuelo.

6.1. Commits relevantes

Hash corto	Mensaje
564090e	Inicializo el proyecto incluyendo la ventana y el fondo.
afb4ade	Agrego el README.
35f132a	Implemento el vuelo del pajarito siguiendo una distribucion normal.
3bfd15d	Integra el vuelo de Barranquin en main.
d2738ab	Agrego el barranco, donde este se mueve pero aun sin control del jugador.
69610dd	Ajusto dispersion del vuelo de Barranquin (prueba en main).
d2b8c71	Añado el control con el teclado, físicas de colisiones y sistema puntaje.
9f6beeb	Ajusta velocidad de control del barranco tras pruebas.
9378996	Ajusto la dispersion del vuelo de Barranquin tras pruebas.
807b58f	Resuelvo el conflicto de dispersion del vuelo: unifica en 60.

Tabla 3: Commits relevantes observados en la evidencia.

Aunque la tabla enumera los commits relevantes visibles en la captura, se debe notar que la evidencia principal para ramas y merges es el grafo local incluido en la siguiente figura.

```
(base) arnalytox@MacBook-Pro-de-arnaldo-9 githu % git log --graph --oneline --all
* 807b58f (HEAD -> main) resuelvo el conflicto de dispersion del vuelo: unifica en 60
| \
| * 9378996 (feature/barranco-control) ajusto la dispersion del vuelo de Barranquin tras pruebas
| * 9f6beeb Ajusta velocidad de control del barranco tras pruebas
| * d2b8c71 añadido el control con el teclado, físicas de colisiones y sistema puntaje
| * d2738ab agrego el barranco, donde este se mueve pero aun sin control del jugador
| * 69610dd ajusto dispersion del vuelo de Barranquin (prueba en main)
| /
| * 3bfd15d Integra el vuelo de Barranquin en main
| \
| * 35f132a (feature/pajaro-random) implemento el vuelo del pájaro siguiendo una distribucion normal
| * afb4ade agrego el README
| /
* 564090e (origin/main) Inicializo el proyecto incluyendo la ventana y el fondo
(base) arnalytox@MacBook-Pro-de-arnaldo-9 githu % git diff --stat HEAD feature/barranco-control -- settings.py
settings.py | 1 -
1 file changed, 1 deletion(-)
```

Figura 1: Evidencia local del historial mediante `git log --graph --oneline --all`.

7. Evidencia del three-way merge

El historial muestra una integracion del vuelo de Barranquin en el commit 3bfd15d. Esta integracion se interpreta como un merge no lineal porque la rama `feature/pajaro-random` tenia su propio avance, mientras que `main` tambien habia cambiado al incorporar el `README.md`. Por lo tanto, Git debio crear un commit de integracion en vez de simplemente avanzar el puntero de la rama principal.

Tambien se observa una segunda linea de trabajo asociada a `feature/barranco-control`. En esa rama se agregaron el barranco, el control con teclado, las colisiones, el puntaje y ajustes de parame-

tros. Esta evidencia muestra un flujo de desarrollo basado en ramas funcionales, no solo una secuencia lineal de commits.

8. Resolucion manual de conflicto

El commit 807b58f, con mensaje *“resuelvo el conflicto de dispersion del vuelo: unifica en 60”*, documenta una resolucion manual de conflicto. El conflicto se produjo porque distintas ramas modificaron un parametro relacionado con la dispersion del vuelo de Barranquin en `settings.py`.

Git no puede resolver automaticamente un conflicto cuando dos lineas de desarrollo cambian la misma linea o una zona cercana del mismo archivo. En este caso, la solucion consistio en escoger un valor unico para la dispersion del vuelo y dejar el archivo en un estado coherente. La captura tambien incluye una salida de `git diff --stat HEAD feature/barranco-control -- settings.py`, donde se observa una diferencia localizada de una linea, lo que es consistente con un conflicto de parametros.

9. Analisis de metricas del codigo actual

Para evaluar el tamano y distribucion del proyecto actual, se revisaron los archivos principales entregados. La mayor parte de la logica se concentra en `main.py`, mientras que `bird.py` y `gate.py` separan las entidades principales del juego.

Archivo	Lineas aprox.	Interpretacion
<code>main.py</code>	267	Archivo principal. Coordina estados, eventos, actualizacion, colisiones, puntaje y dibujo.
<code>bird.py</code>	123	Contiene la logica del pajarito autonomo y su representacion visual.
<code>gate.py</code>	93	Implementa el barranco, control, dificultad y colisiones.
<code>settings.py</code>	84	Centraliza parametros de configuracion y paleta de colores.
<code>utils.py</code>	14	Archivo pequeno con utilidad compartida.
<code>README.md</code>	86	Documentacion de instalacion, uso y estructura.
<code>requirements.txt</code>	1	Dependencia principal: Pygame.

Tabla 4: Tamano aproximado de archivos actuales.

9.1. Interpretacion de metricas

- **Hotspot principal:** `main.py`, porque concentra el bucle del juego y la coordinacion de estados.
- **Separacion modular:** `bird.py` y `gate.py` evitan que toda la logica quede en un unico archivo.
- **Configuracion centralizada:** `settings.py` permite cambiar dificultad, fisica y colores sin tocar las clases principales.

- **Trazabilidad:** los commits muestran avances funcionales pequenos, lo que facilita explicar que se agrego en cada etapa.

10. Coherencia con el README

El `README.md` actual describe correctamente que el proyecto es una variante invertida de *Flappy Bird*: el jugador controla el barranco y Barranquin vuela solo. Tambien informa los requisitos, la ejecucion con `python3 main.py` y la dependencia `pygame>=2.5.0`.

Como observacion menor, conviene verificar que el comando de clonacion del `README.md` apunte exactamente al repositorio final: <https://github.com/nalytox/bird-flappy>. Esto no afecta el funcionamiento del codigo, pero mejora la consistencia documental de la entrega.

11. Reflexion final

Durante este proyecto aprendi que Git no sirve solamente para guardar versiones, sino tambien para dejar evidencia del proceso de desarrollo. Separar el trabajo en ramas permitio avanzar por funcionalidades, primero con el vuelo de Barranquin y luego con el barranco, el control y las colisiones. Esto hizo mas claro el historial y facilito explicar que parte del juego se construyo en cada etapa.

Tambien pude observar la importancia de los commits descriptivos. Mensajes como *“implemento el vuelo del pajaro siguiendo una distribucion normal”* o *“anado el control con el teclado, fisicas de colisiones y sistema puntaje”* permiten entender el avance sin abrir todos los archivos. En cambio, si todos los commits tuvieran nombres genericos, el historial no serviria como evidencia de aprendizaje.

La resolucion del conflicto en `settings.py` fue especialmente relevante porque muestra una situacion real del uso de Git: dos ramas pueden modificar un mismo parametro y Git requiere una decision manual. En este caso, la solucion consistio en unificar el valor de dispersion del vuelo, dejando un criterio unico para el comportamiento de Barranquin.

12. Declaracion de uso de inteligencia artificial

Se utilizo inteligencia artificial como apoyo para estructurar ideas, revisar el enfoque del informe, orientar la organizacion del codigo y redactar documentacion. Tambien se uso como apoyo para transformar la idea del juego en una estructura modular de Python con Pygame.

Sin embargo, el estudiante declara que reviso el codigo entregado y que esta en condiciones de explicar sus componentes principales: el bucle de juego en `main.py`, el movimiento de Barranquin en `bird.py`, el barranco en `gate.py`, la configuracion en `settings.py`, el uso de ramas, el merge y la resolucion manual del conflicto. Los commits, la subida al repositorio y la evidencia local del historial corresponden al proceso de trabajo del estudiante.

13. Conclusion

El repositorio actualizado respalda una version funcional del proyecto **Flappy Inverso**. A diferencia de la revision anterior, ahora el codigo entregado contiene los modulos necesarios para el pajaro, el barranco, el control del jugador, las colisiones, el puntaje y la dificultad progresiva. El informe queda alineado con el codigo actual y con la evidencia local de Git presentada.